Building AI Agents for PV Creation & Approval

Hands-on Workshop

AI Agent Bootcamp — Komatsu, 30 March – 3 April 2026
Teerasej Jiraphatchandej, Microsoft MVP AI
www.youtube.com/teerasej , www.nextflow.in.th, www.facebook.com/teerasej

The Business Problem

Payment Voucher requests are manual, error-prone, and slow to approve.

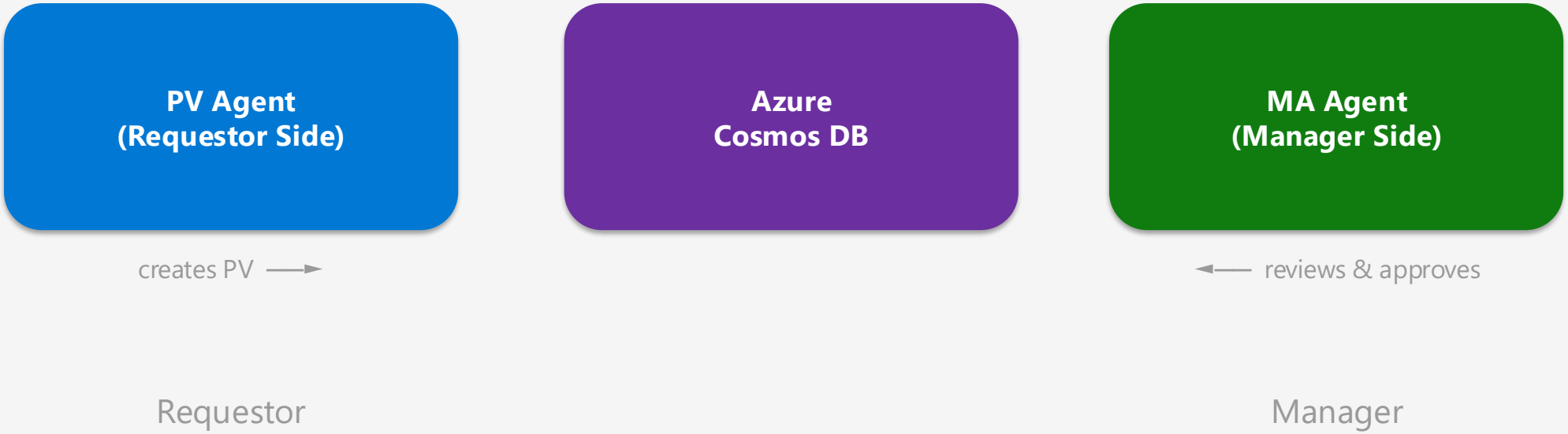
Manual data entry — requestors fill forms by hand, missing fields are common

No validation — incomplete or incorrect PVs reach the manager's desk

Slow approval chain — managers manually search through submissions

No single source of truth — data scattered across emails and spreadsheets

The Solution: Two AI Agents



We'll build two agents — one for creation, one for review. They share a database.

What is an AI Agent?

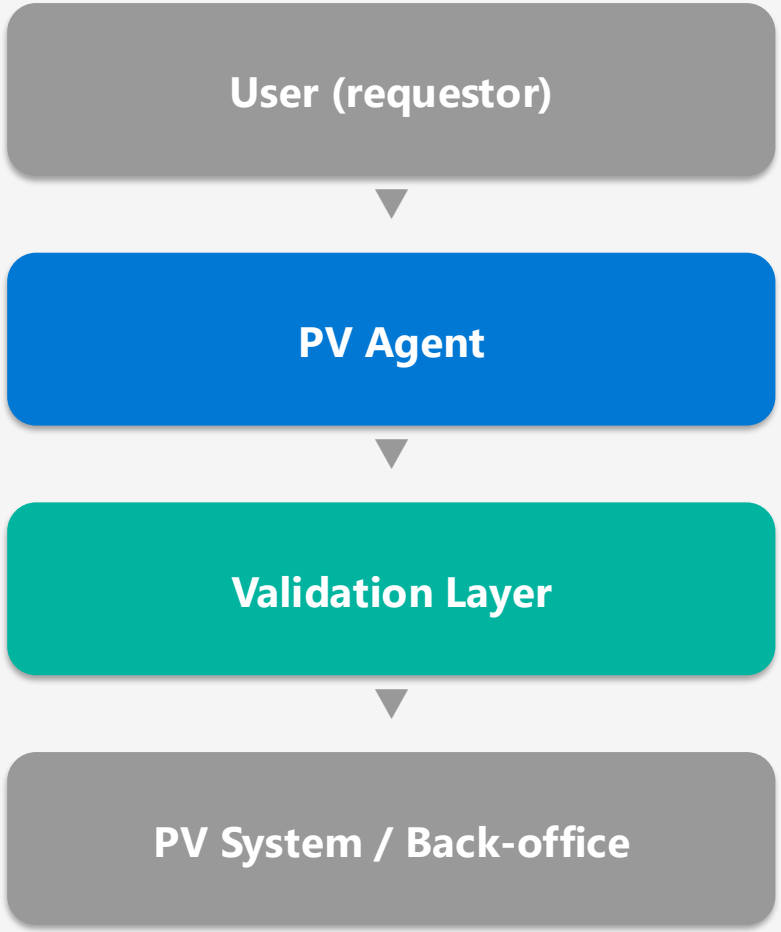
AI Agent = LLM + Instructions + Tools

LLM
(the brain)

Instructions
(the rules)

Tools
(the hands)

PV Agent Boundaries



The agent is a data assistant. It does NOT approve, does NOT access finance systems.

LET'S DO IT

D3-M1: Environment Setup

Create a Microsoft Foundry project
Deploy GPT-4.1 model
Configure .env and run smoke test

The Agent Instruction is Everything

It defines who the agent is, what it knows, and what it refuses to do.

Anatomy of a Good Instruction

1. Role	"You are PV Agent..."
2. Required Fields	List every field the agent must collect
3. Constraints	What the agent must NOT do
4. Output Format	The exact JSON template

Slot-Filling Conversation Pattern

The agent follows a slot-filling pattern — collect one field at a time, never invent data.

Instruction Design Rules

- ✓ Never ask for a field the user already provided
- ✓ Never invent data that the user did not provide
- ✓ State assumptions explicitly (e.g. "I'll use THB as the default currency")
- ✓ Only output the final JSON after user confirms the summary

Agent Framework SDK — Minimal Pattern


```
agent = Agent(
    client   = AzureOpenAIResponsesClient(...),
    instructions = PV_AGENT_INSTRUCTIONS,
    tools    = [...]
)

# Conversation loop
while True:
    user_input = input("You: ")
    response = await agent.run([user_input])
    print(response)
```

Three ingredients: client connection, instruction string, conversation loop.

LET'S DO IT

D3-M2: Agent Role & Conversation Design

Write the PV Agent instruction (system prompt)

Test 3 user stories in Foundry playground

Implement the agent in Python with Agent Framework SDK

New Requirement: Budget Type

Finance says: every PV now needs a budgetType — either Expense or Investment.

Expense

Subscriptions, fees, training registrations, consumable items

Investment

Equipment purchases, tool licenses, infrastructure upgrades

Instruction Grounding vs. Code Logic

Code approach

Keyword list → if/else logic
Misses edge cases
"one-time monitoring upgrade" → ???

Instruction approach

Add rule to the prompt
LLM understands nuance
Handles ambiguity naturally

If the decision depends on understanding meaning, put it in the instruction — not code.

What Changes: Instruction Only

Zero code changes. Only the instruction string is updated.

REQUIRED FIELDS section — add:

```
- expense.budgetType: MUST be "Expense" or "Investment"
    - "Expense" for recurring/operational costs
    - "Investment" for one-time capital expenditures
```

The cheapest, fastest way to extend agent output.

LET'S DO IT

D3-M3: Data Grounding via Instruction

Add budgetType classification rule to instruction

Update OUTPUT FORMAT with new field

Test with 3 user stories (Expense, Investment, Ambiguous)

When Instructions Aren't Enough

Instructions handle rules.
But the agent can't look up facts from thin air.

Example:
User: "Project is IT Internal"
Agent needs: budget = 100,000 THB, remaining = 100,000 THB
→ The agent doesn't know this. It needs a data source.

Instruction vs. Tool — Decision Matrix

Classification rules, enums

→ Instruction grounding

Data lookup, calculations, I/O

→ Function tool

The @tool Decorator Pattern

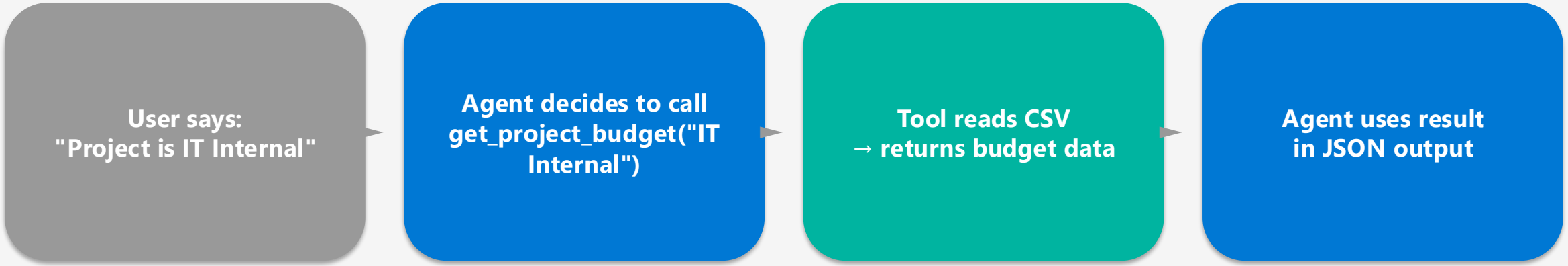

```
@tool(approval_mode="never_require")
def get_project_budget(
    project_name: Annotated[str, Field(
        description="The name of the project to look up"
    )]
) -> str:
    """Look up budget from CSV data file."""
    ...
```

① @tool — marks this as an agent capability

② Annotated + Field — tells the agent what each parameter means

③ Return str — agent reads the result as conversation context

Data Flow: Agent Calls Tool



The agent autonomously decides when to call the tool. You don't trigger it manually.

LET'S DO IT

D3-M4: Function Tool for Budget Lookup

Implement get_project_budget with @tool decorator

Register tool with Agent via tools=[...]

Test with known & unknown project names

Reversed Data Flow

Exercise 4 (Read)
Tool → returns data → Agent

Exercise 5 (Write)
Agent → passes JSON → Tool

Now the agent is the sender.
It assembles the complete PV payload and hands it off to the tool.

The Completeness Gate

Instruction rules that prevent premature submission:

1. Do NOT call submit_pv until the user confirms AND all fields are filled
2. Status must be "ReadyForSubmission" before calling the tool
3. Never submit with placeholders, empty strings, or zero amounts
4. Never submit a PV with status = "Draft"

The instruction acts as a gate — preventing premature submission.

Stub First, Wire Later

The submit tool just prints JSON to the terminal. No database yet.

```
--- PV Submission Received ---
{ "pv": { "pvTitle": "...", ... } }
--- End of PV Submission ---
```

Validate the payload shape before adding infrastructure complexity.

LET'S DO IT

D3-M5: Submit PV Tool

Implement submit_pv with @tool decorator
Add completeness gate rules to instruction
Test full flow: collect → confirm → submit

From Stub to Real Database

Before
submit_pv → print(json)
(terminal only)

⟹

After
submit_pv → CosmosClient
→ Cosmos DB

Same tool interface. Only the function body changes. The agent doesn't notice the swap.

Why Cosmos DB Serverless?

Pay-per-request — no minimum throughput charge

NoSQL — stores JSON natively (matches PV structure perfectly)

Shared container — both PV Agent and MA Agent use the same data

Partition key: /id — each document is its own partition (UUID)

Separation of Concerns

Agent Instruction	unchanged
Tool Interface	unchanged — submit_pv(pv_json: str) -> str
Tool Implementation	CHANGED — Cosmos DB insert

Clean separation. The agent's behavior is stable even as the backend evolves.

LET'S DO IT

D3-M6: Store PV in Cosmos DB

Provision Cosmos DB Serverless account

Replace submit_pv body with Cosmos DB insertion

Verify the document in Data Explorer

PV Agent: What We Built

6 layers, incrementally added — one concept per exercise.

Two Sides of the Same Workflow

PV Agent
(completed ✓)

Azure
Cosmos DB

MA Agent
(building now)

Now we build the other side.

Same Patterns, Different Role

	PV Agent	MA Agent
Role	Collects & creates PV	Reviews & approves PV
Data direction	Writes to Cosmos DB	Reads from Cosmos DB
SDK pattern	Same	Same
Instruction	Different	Different

Agent specialization = same framework, different instructions.

The Manager's Instruction

Key differences from PV Agent:

Role: "Help managers review and explore PV requests" (not create)

Understands the PV JSON structure (reads the data contract)

Constraint: "Do NOT modify, do NOT invent data"

Constraint: "Do NOT approve or reject" (in this exercise — tools come next)

Focus: Summarize, flag issues, answer questions about PV content

Reusing the SDK Pattern


```
                                agent = Agent(
    client   = AzureOpenAIResponsesClient(...),
    instructions = MA_AGENT_INSTRUCTIONS,  # ← only this changes
    tools    = [...]                        # ← added later
)

# Same conversation loop as PV Agent
```

Identical code structure. Only the instruction text changes.

LET'S DO IT

D4-M1: MA Agent Setup & Foundation

Write the MA Agent instruction for the reviewer role

Implement conversation loop with Agent Framework SDK

Test with manager review scenarios (paste PV JSON, ask questions)

A Manager Needs Two Actions

Query

"Show me all pending PV requests"

→ get_pv_requests(status)

Act

"Approve PV pv-001"

→ update_pv_approval_status(id, status)

One agent, two tools — read and write.

Prototype with In-Memory Data

Before connecting to Cosmos DB, use a Python list as the data source.

```
SAMPLE_PV_DATA = [
    {"id": "pv-001", "pvTitle": "GitHub Copilot",
     "approval": {"status": "Pending"}, ...},
    {"id": "pv-002", "pvTitle": "Conference Fee",
     "approval": {"status": "Approved"}, ...},
    {"id": "pv-003", "pvTitle": "Ergonomic Chairs",
     "approval": {"status": "Approved"}, ...},
]
```

Prototype fast. Validate tool patterns and agent behavior without database complexity.

Registering Multiple Tools


```
tools=[get_pv_requests, update_pv_approval_status]
```

The agent decides which tool to call based on the user's question.
"Show me pending PVs" → get_pv_requests
"Approve pv-001" → update_pv_approval_status

LET'S DO IT

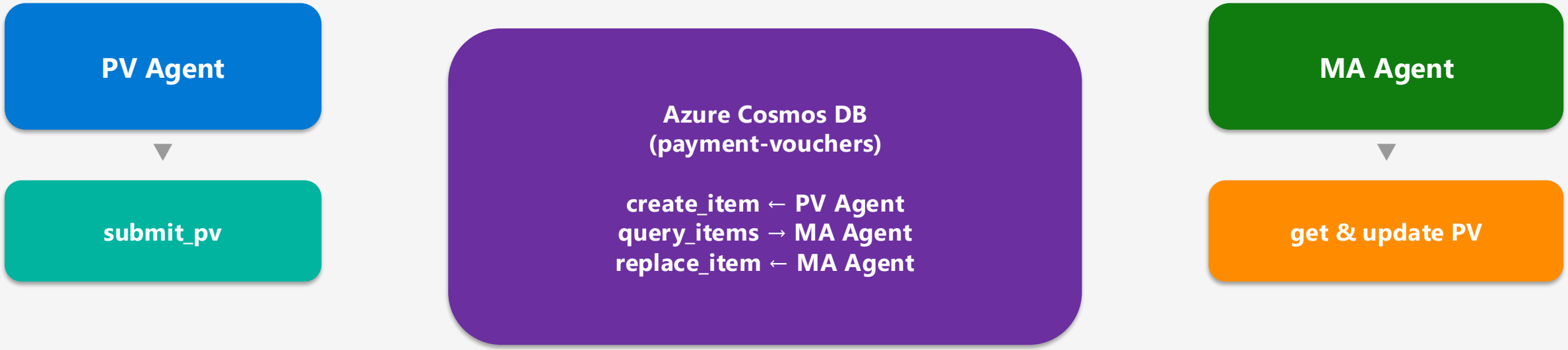
D4-M2: Function Tools for PV Review & Approval

Implement get_pv_requests and update_pv_approval_status

Register both tools with Agent

Test: query pending → approve → verify approval

The Full Pipeline — End to End



Both agents now share live data. Submit a PV, then review and approve it.

What Changed: Implementation Only

Agent Instruction	unchanged
Tool Interface	unchanged — same function signatures
Tool Implementation	CHANGED — SAMPLE_PV_DATA → CosmosClient

Same pattern as PV Agent's Cosmos upgrade. The swap is always in the implementation layer.

LET'S DO IT

D4-M3: Connect MA Agent to Cosmos DB

Add get_cosmos_container() helper function

Replace both tools with Cosmos DB operations

End-to-end: PV Agent submit → MA Agent query → approve → verify

What We Built Today

PV Agent
6 exercises

Cosmos DB
shared container

MA Agent
3 exercises

1. System Prompt Design
2. Instruction Grounding
3. Function Tools (Read)
4. Function Tools (Write)
5. Cloud Persistence
6. Agent Specialization

Key Takeaways

1. **The system prompt is your most powerful tool**
2. **Classification → instruction. Facts → function tool.**
3. **Stub first, wire the database later**
4. **Same SDK + different instruction = different agent personality**
5. **Agents specialize — separate creation from review**